

Grade Change Intelligence — Complete Project Documentation

Honeywell Hackathon Submission

1. The Problem, In Full

Honeywell's Quality Control System (QCS) already runs Multivariable Model Predictive Control (MPC) for grade changes on paper machines — it calculates targets, trajectories, and coordinates ramping of stock flow, filler flow, dryer steam pressure, and machine speed. It works, but two gaps remain:

1. **Grade changes still cause losses.** Even with good execution, mills produce off-spec, broke, or cull material while variables stabilize after a transition.
2. **The system executes but doesn't learn or explain.** It doesn't use historical data to predict trouble before it happens, and it gives no plain guidance to operators — which matters more now that experienced operator intuition is increasingly scarce.

What was asked for (verbatim requirements, organized)

Predict: whether Basis Weight will deviate more than **2.5%** from setpoint during a grade/recipe change, before quality limits are exceeded.

Recommend: setpoints that keep the system in safe operating limits.

Reduce: stabilization time — identify which loops/parameters most affect it, and what setpoints would speed it up.

Explain: rationale behind every prediction/recommendation.

Deliverables:

- A working solution
- Documentation of building blocks, with communication between modules explained
- A dashboard showing new correlations found + their impact + a future-state projection if current trends continue, with suggested setpoints
- Dashboard also showing which loops/parameters most affect stabilization + suggestions
- Every suggestion tagged with its source

- Accept/Reject UI, with responses logged (their built-in mechanism for evaluating suggestion quality over time)
- A filled-in presentation using their template
- Everything converted to PDF or zipped for upload

Requirement Compliance Matrix (proof nothing was missed)

The 4 Hackathon Challenge points, verbatim, mapped to the build

#	Their exact requirement	What satisfies it	Where
1	<i>"Predict when the system specification for main variable Basis Weight is at risk of going off-spec (deviates more than 2.5% from the setpoint) during transition of grades... and recommend corrective action before quality limits are exceeded."</i>	RandomForestClassifier trained on early-window (first 5 min) features, 91.4% test accuracy, 0.973 ROC-AUC, 0% false-negative rate. Predicts risk before the 18–24 min settling window, then triggers setpoint recommendations if at-risk.	Tab 1 (Model Performance) + Tab 4 (Live Risk & Recommendations)
2	<i>"Provide recommendation of setpoints to keep the system in safe operating limits."</i>	<code>recommend_setpoints()</code> — nudges the top-importance CONTROLLABLE variables (stock flow, filler flow, steam pressure, machine speed) toward the historical average of safe events of the same grade transition	Tab 4, recommendation cards
3	<i>"Reduce stabilization time to reach steady state."</i>	<code>stabilization_drivers()</code> — separately correlates controllable variables against <code>settle_t</code> (minutes to reach steady state), ranks which speed it up vs drag it out	Tab 3 (Stabilization Analysis)

#	Their exact requirement	What satisfies it	Where
4	<i>"Provide rationale behind the prediction/recommendation."</i>	<code>rationale_for()</code> — every suggestion gets a plain-English, source-tagged sentence; feature importances double as the model's own rationale	Every recommendation card + Tab 1 feature-importance chart

The stated Constraints, mapped

Their constraint	How it's met
<i>"Use process limits from recipe, historical operator actions data, historical trends and trajectories during failures and success scenarios."</i>	<code>generate_data.py</code> simulates 140 grade-change events with realistic recipe-driven settling dynamics; <code>recommend_setpoints()</code> explicitly compares an at-risk event against the historical trajectories of <i>safe</i> events of the same grade transition
<i>"Use historical data to find correlations in various loops/system parameters like Stock flow, filler flow, steam pressure, machine speed, moisture, ash, caliper, recipe limits."</i>	<code>discover_correlations()</code> computes Pearson correlation for every one of these variables against deviation; <code>stabilization_drivers()</code> does the same against settle time
<i>"Find new correlations not defined in the system but may have impacted the process."</i>	Every correlation is explicitly tagged "Known (in QCS recipe/spec logic)" or "Newly discovered (not in standard recipe/spec)" — ash, caliper, and recipe headroom all surfaced as newly discovered in this dataset

The 6 Expectations/Deliverables, mapped

#	Requirement	Status	Where
1	Develop a solution to solve the challenge	✓	<code>app.py</code> , <code>core.py</code> , <code>generate_data.py</code>
2	Document the building blocks clearly, showing	✓	This document, Sections 2–6

#	Requirement	Status	Where
	communication between modules, brief explanation		
3	Dashboard: new correlations found + impact + future state if deviations follow current trend/trajectory + suggested setpoints	✓	Tab 2 (correlations) + Tab 4 (calibrated settle-time projection = the "future state if trend continues" requirement, ± 2.05 min backtest accuracy)
4	Dashboard: loops/parameters causing high impact on stabilization + setpoints to stabilize faster, based on historical data	✓	Tab 3
5	Tag every suggestion with possible source of inference (historical data, recipe, etc.)	✓	<code>rationale_for()</code> , shown on every card
6	Accept/Reject UI; responses recorded to evaluate suggestion quality/accuracy over time	✓	Tab 4 buttons → SQLite <code>db/feedback.db</code> → Tab 5 (Feedback Log) with acceptance-rate KPI
—	Submit a presentation using their template	✗ Not built — send the template's slide titles or upload it	
—	Convert to PDF or zip, upload via "Upload Files"	✓ <code>gradechange-solution.zip</code> ready; presentation still needs export to PDF once written	

Bottom line: every technical requirement in the brief has a corresponding, working, tested piece of the solution. The only open item is the presentation file itself.

2. Solution Overview

One sentence: a Streamlit dashboard, backed by a RandomForest risk model and a set of statistical analyses, that predicts deviation risk from the first 5 minutes of a grade change, explains why, recommends operator-controllable corrective setpoints, separately ranks what drives stabilization time, projects when a transition will settle, and logs every operator accept/reject decision for future quality tracking.

Stack: Python, pandas, scikit-learn, Streamlit, Plotly, SQLite. Nothing external — no API keys, no network dependency at runtime, so nothing can fail from a bad connection mid-demo.

Why no LLM / no time-series deep learning: both were considered (see the original five-idea brainstorm) and deliberately dropped for a hackathon under a hard time budget — they add failure points (API latency/cost, training instability) without adding to the deliverable checklist. Every requirement is met with simpler, fully-explainable methods, which is itself a defensible design decision worth stating to judges directly.

3. Architecture — module by module

generate_data.py

Synthetic paper-mill dataset generator. 140 grade-change events, minute-by-minute, with exponential settling dynamics, grade-appropriate overshoot, and THREE deliberately-planted "hidden" relationships not in any standard recipe/spec: ash content, caliper, and recipe headroom. These exist so the correlation-discovery step in the dashboard has something genuine, verifiable to find -- rather than fabricating "discoveries" against inert noise.

-> data/process_data.csv

core.py (all business logic; testable independently of the UI)

load_events()	CSV -> DataFrame
build_features()	per-minute data -> one row per event: - early-window (t=0-5 min) mean + slope for every process variable ("we just started, predict now") - label_off_spec: still >2.5% off-target at t=18-24 min - settle_t: first minute after which the event NEVER again exceeds the 2.5% band (computed via a reversed-cumulative-max over the exceeds series)
discover_correlations()	Pearson correlation of every early-window variable against late-stage deviation; splits results into "Known (in QCS recipe/spec logic)" vs "Newly discovered"
stabilization_drivers()	same idea, different target: correlates CONTROLLABLE variables against settle_t specifically -- this is a separate analysis from deviation-risk, because the brief

`train_model()`

asks **for** stabilization-time drivers **as** their own thing
RandomForestClassifier, **300** trees, depth **6**, **75/25**
stratified train/test split. **Returns** accuracy, ROC-AUC,
AND a **full** confusion matrix **with** **false-negative** /
false-positive rates broken **out** explicitly

`rationale_for()`

turns a feature name **into** a plain-English, source-tagged
sentence ("Source: historical grade-change data -- early
trend of steam pressure (recipe-defined driver), ranked
#2 by model importance")

`recommend_setpoints()`

for an **at-risk** event, nudges its top-N important features
toward the historical mean **of** SAFE events **of** the same
grade transition -- restricted to CONTROLLABLE_FEATURES
only (stock flow, filler flow, steam pressure, machine
speed), never quality outputs **or** fixed properties

`project_settle_time()`

fits a line through the early-**window** deviation trend **and**
extrapolates **to** find **when** it crosses the spec band --
this **is** the quantitative "future state if trend
continues" claim

`fit_settle_calibration()`

a naive linear extrapolation **of** a fast **initial** descent
systematically UNDER-estimates **true** settle **time**, because
real settling **is** asymptotic, **not** linear (measured: **~14**
minute average bias **on** this dataset). This **function**
fits a correction (actual **~=** m * raw_projection + c)
against historical outcomes, cutting average error **from**
14.4 minutes **to** **2.05** minutes

`calibration_backtest_mae()`

reports that **2.05-minute** figure **for** display, so the
dashboard's credibility claim is measured, not asserted

`app.py` (Streamlit dashboard, presentation-only -- imports `core.py` for everything else)

5 tabs: Model Performance | Correlation Discovery | Stabilization Analysis |

Live Risk & Recommendations | Feedback Log

SQLite-backed accept/reject logging (db/feedback.db)

Custom dark "instrument panel" visual theme (see Section 5)

`README.md`

architecture summary (shorter version of this document)

`.streamlit/config.toml`

base theme colors

Data flow in one line: `generate_data.py` → `process_data.csv` → `core.build_features()` →
everything else in `core.py` operates on that single feature table → `app.py` renders it.

4. What each dashboard tab shows, and why it exists

Tab 1 — Model Performance

- Accuracy (91.4%), ROC-AUC (0.97), event count, **false-negative rate (0.0%)** as a headline KPI — not just accuracy, because in a control-system context a false negative (model says "safe," process actually goes off-spec) is the costly error, not a missed percentage point of accuracy.
- Feature importance chart — this **is** the model's rationale mechanism. Every downstream prediction is explained by these same weights; nothing is a black box.
- Confusion matrix + a plain-English callout of exactly how many false negatives vs false positives occurred in testing, and why the asymmetry matters.

Tab 2 — Correlation Discovery

- Full correlation table, each row tagged **Known** (already in QCS logic) or **Newly discovered** (not in standard recipe/spec), color-coded.
- A dedicated callout list of only the newly-discovered ones: in this dataset, **ash content** ($r=-0.488$), **caliper** ($r=-0.483$), and **recipe headroom** ($r=0.457$) — none of which a standard MPC recipe currently accounts for.

Tab 3 — Stabilization Analysis (*new — closes a gap the deliverable calls out separately from risk prediction*)

- Ranks operator-controllable variables by correlation with **settle_t** specifically (not deviation magnitude). Top drivers found: machine speed (higher = slower stabilization, $r=0.426$), steam pressure and stock flow (higher = faster, $r=-0.42$, -0.41).
- Directly answers "what setpoints help stabilize the system faster" with a ranked, actionable list, separate from the risk-prediction recommendations in Tab 4.

Tab 4 — Live Risk & Recommendations

- Pick any of the 140 events; get a live risk gauge (RandomForest probability), the deviation trajectory chart with the $\pm 2.5\%$ spec band marked, and a **calibrated settle time projection** ("based on the first 5 minutes, projected to come within spec by minute X, ± 2.05 minutes average backtest error").
- Setpoint recommendations, each with a source-tagged rationale, and **Accept/Reject buttons** that write to SQLite immediately.

Tab 5 — Feedback Log

- Every logged response, an acceptance-rate KPI, and an explicit note tying it back to purpose: this is how Honeywell would measure and improve suggestion quality over time — the log's value is longitudinal, not just a UI nicety.
-

5. Design decisions worth explaining if a judge asks

"Is this real ML, or just a formula?" — The risk prediction is a properly trained/ tested RandomForestClassifier (91.4% test accuracy, 0.97 AUC, trained on 105 events, tested on 35 it never saw). The 2.5%-deviation *label definition* used to train it is a straightforward business rule taken directly from Honeywell's own spec — that's not the same thing as the model itself being a formula. Correlation Discovery is deliberately **not** ML — it's Pearson correlation, used on purpose because the brief asks for correlation discovery as a distinct capability from prediction.

"Why RandomForest and not a neural net?" — Feature importances come for free and are directly usable as rationale, which the brief explicitly requires ("tag every suggestion with source of inference"). A more complex model would need a separate explainability layer (e.g. SHAP) to get the same guarantee, for no accuracy benefit on a ~140-event dataset.

"Why synthetic data?" — No historical dataset was provided in the problem statement at preview time. `generate_data.py` produces physically-motivated (not random) data: real exponential settling curves, grade-appropriate overshoot, and intentionally planted hidden correlations so discovery has something real to verify against. Swapping in a real Honeywell dataset requires changing exactly one line — the path in `core.load_events()` — because every downstream function only assumes the column schema, not the data source.

"Why does the settle-time projection get calibrated?" — Because we checked, and a raw linear extrapolation of the early trend was systematically wrong (14.4 min average error) in a predictable direction. Rather than presenting an inaccurate number confidently, we measured the bias and corrected for it, then reported the post-correction error (2.05 min) honestly instead of hiding it. This kind of self-checking is a stronger signal of engineering maturity than a number that merely looks precise.

6. Visual design

The dashboard intentionally does **not** look like a generic analytics template. It's styled as a dark industrial instrument panel / HMI console — the actual visual language of real DCS/QCS control-room screens (Honeywell Experion and similar systems), not a decorative choice:

- **Palette:** near-black canvas (#0B1420) with panel surfaces (#131F2E), and a functional status-color system rather than one decorative accent: teal (#2DD4C8) for primary data, green (#34D399) for safe, amber (#F5A623) for caution, red (#E5484D) for alarm — the same convention real alarm-management HMI screens use.
- **Typography:** IBM Plex Sans for headers/body, IBM Plex Mono for all numeric readouts — chosen for its technical/engineering heritage rather than a generic UI font. Degrades

gracefully to system sans-serif if the machine has no internet access to fetch the font.

- The risk-prediction tab centers on an **analog gauge chart** (Plotly `Indicator`, green/amber/red zones) instead of a plain number or progress bar — a deliberate nod to a real physical control-panel gauge, and the one "signature" visual element the page is built around.
- Every panel, badge, and callout box uses one consistent custom component system (`.hmi-panel`, `.hmi-badge`) rather than default Streamlit alert boxes, so the whole dashboard reads as one designed system rather than a stock template.

7. How to run

```
pip install streamlit scikit-learn pandas plotly
python3 generate_data.py      # regenerates data/process_data.csv (only needed once)
streamlit run app.py
```

To point at a real dataset instead of the synthetic one: edit the `path` default in `core.load_events()`, keeping the same column schema (see `generate_data.py`'s docstring for the exact columns expected).

8. Current metrics (synthetic dataset, will change with real data)

Metric	Value
Model accuracy (test set)	91.4%
ROC-AUC	0.973
False negative rate	0.0% (0 of 21 true-risk test cases missed)
False positive rate	21.4% (3 of 14 safe test cases over-flagged)
Events in dataset	140
Top newly-discovered correlations	ash ($r=-0.488$), caliper ($r=-0.483$), recipe headroom ($r=0.457$)

Metric	Value
Top stabilization drivers	machine speed ($r=0.426$, slower), steam pressure ($r=-0.421$, faster), stock flow ($r=-0.412$, faster)
Settle-time projection accuracy	± 2.05 min average error (calibrated), vs ± 14.4 min uncalibrated

9. Honest limitations (better to say these than have a judge find them)

- Trained and evaluated on synthetic data — real accuracy on Honeywell's actual process data is unknown until tested, though the pipeline itself doesn't change.
 - 140 events / 35 test cases is a small test set — the 0% false-negative rate is a good sign but not a statistically strong guarantee at this sample size.
 - The settle-time projection is a calibrated linear extrapolation, not a full process model — it's transparent and auditable by design, but won't capture more complex non-monotonic dynamics as well as a proper dynamical model would.
 - Setpoint recommendations are based on historical averages of similar safe transitions, not a constrained optimizer — a reasonable, explainable v1, with a real MPC-style optimization as a natural next iteration.
-

10. Presentation talking points (map directly to your slide template)

1. **Problem** — grade changes still cause losses despite good MPC execution; the gap is learning and explainability, not control execution.
2. **Approach** — three purpose-built tools, not one model doing everything: statistics for correlation discovery, ML for risk prediction, a calibrated trend model for settle-time projection. State this explicitly — it preempts "is this real ML?" questions.
3. **Architecture** — walk through Section 3 above; emphasize the one-line dataset swap for real data.
4. **Results** — lead with the false-negative rate (0%), not just accuracy; it's the more relevant metric for the audience.
5. **Demo** — screenshot each of the 5 tabs in action; judges remember demos more than diagrams.

6. **Feedback loop** — explicitly state the Accept/Reject log is how Honeywell would measure and improve suggestion quality in production, not just a UI feature.
7. **Honesty close** — one sentence on synthetic data and the one-line swap path. Shows engineering maturity instead of looking evasive.

11. What To Actually Upload (submission checklist)

Their instructions: "Convert to PDF or Zip all the files and upload below... All files must be in pdf or zip file formats only."

You upload 2 files total:

#	File	Covers
1	gradechange-solution.zip	Working solution, all documentation (this file + README), and the dashboard code
2	presentation.pdf	Their required slide template, filled in with real content and dashboard screenshots

Deliverable-by-deliverable mapping:

Their requirement	Where it lives	Status
1. Develop a solution to solve the challenge	app.py + core.py + generate_data.py	✓ Done
2. Document building blocks, module communication, brief explanation	This file (Sections 1–6)	✓ Done
3. Dashboard: correlations + impact + future-state trend + suggested setpoints	Tab 2 + Tab 4 in app.py	✓ Done — screenshot for the presentation
4. Dashboard: stabilization drivers + faster-settling setpoints	Tab 3 in app.py	✓ Done — screenshot for the presentation

Their requirement	Where it lives	Status
5. Every suggestion tagged with source of inference	<code>rationale_for()</code> in <code>core.py</code>	✓ Done
6. Accept/Reject UI, responses recorded	Tab 4 buttons + <code>db/feedback.db</code> + Tab 5	✓ Done
7. Presentation using their template	—	✗ Not started — send the template's slide titles or upload it, and the content gets written next

Before final zip: click Accept/Reject on at least one recommendation locally so `db/feedback.db` isn't empty when you package — a populated log is stronger proof the mechanism works end to end than an empty one.